

Elderly Fall Detection on ESP32 — 20-Minute Presentation Script

How to use this 20-minute script

This script is designed for a project defense, class presentation, or technical demonstration. Speak slowly. Do not read every word mechanically. The time labels are approximate. The live demonstration should be shown after the hardware and system sections.

0:00–1:30 — Opening

Good morning everyone. My project is called Pre-Impact Fall Detection on ESP32. The goal is to build a low-cost wearable system that can recognize dangerous fall-related motion and give an alert using an LED and a buzzer. The main idea is simple: instead of waiting until a person hits the ground, can we identify a dangerous movement pattern early enough to warn someone?

1:30–3:00 — The real problem

Falls are a serious safety problem, especially for older adults. A post-impact alarm can still be useful, but it may be too late to prevent injury. A pre-impact warning can support a faster response. However, the system must also avoid alarming during normal activities such as walking, sitting, standing, turning, or climbing stairs. This creates a difficult balance between sensitivity and false alarms.

3:00–4:20 — Project design idea

The project uses one small inertial sensor placed near the waist. The MPU6050 measures acceleration on three axes and angular velocity on three axes. The ESP32 reads those six values, keeps a rolling history of movement, and runs a compact neural network locally. The result is converted into three probabilities: normal movement, alert, and fall. When the decision crosses the selected operating point, the LED and buzzer are activated.

4:20–5:50 — Why use an ESP32

The ESP32 is inexpensive, widely available, small, and powerful enough for TinyML inference. It also has Wi-Fi and Bluetooth for future extensions. The model does not need to send raw sensor data to the cloud. The decision is made locally. This makes the system faster, more private, and potentially usable even when there is no internet connection.

5:50–7:30 — Datasets

The model was developed using public datasets: SisFall Enhanced, KFall, FallAIID, and UMAFall. These datasets contain daily activities and simulated falls collected under different conditions. KFall is especially important because it provides fall onset and impact information. The data sources are not identical, so the project does not only report one accuracy number. It also studies what happens when a model meets a dataset it has never seen.

7:30–9:00 — Data preparation

Every dataset has different file formats, sampling rates, sensor orientations, and labels. The project converts them into a common six-channel representation: ax, ay, az, gx, gy, gz. The working rate is 50 Hz. A two-second window contains 100 samples. The stride is 25 samples, so the model makes a new decision every half second. This same preprocessing contract must be used during training and on the ESP32.

9:00–10:20 — Model architecture

The model is a compact separable CNN. It uses convolution layers to learn time patterns from the six sensor channels. Separable convolutions reduce the number of parameters. Global average pooling prevents the final dense layer from becoming too large. The final output has three classes: non-fall, alert, and fall. The model is deliberately small because the target device is a microcontroller, not a desktop GPU.

10:20–11:40 — Training and quantization

The model was trained on Kaggle using a GPU. After training, it was converted from FP32 to full integer INT8. Quantization reduces memory and makes microcontroller inference practical. The converted model was tested on the desktop before deployment. The model file was then converted into a C header so that it could be stored in ESP32 flash memory.

11:40–13:00 — Physical circuit

The physical circuit uses one primary MPU6050. VCC goes to ESP32 3V3, GND goes to GND, SDA goes to GPIO21, and SCL goes to GPIO22. AD0 is connected to GND, so the I2C address is 0x68. The active buzzer is connected to GPIO4. The LED is connected to GPIO2 through a 220 ohm resistor. For the first physical test, the board is powered only by USB. A battery and TP4056 are optional later.

13:00–14:10 — First hardware test

Before using AI, I tested the physical sensor using a direct-register MPU6050 reader. The serial output showed an I2C device at address 0x68, and the acceleration magnitude while stationary stayed close to 1 G. Moving the board changed the acceleration and gyroscope values. This confirmed that the wiring, I2C communication, sensor reading, LED, and buzzer path were working independently of the neural network.

14:10–15:20 — AI firmware operation

The final AI firmware collects the live samples at 50 Hz. It builds a 100 by 6 window. It applies the same per-window normalization used by the final model. It then quantizes the values to INT8 and invokes TensorFlow Lite Micro. The output is printed as p_bkg, p_alert, and p_fall. The firmware can require multiple consecutive confident windows before activating the alarm, which reduces isolated false positives.

15:20–16:30 — Live demonstration

Now I will demonstrate the system. First, I keep the sensor still. The background probability should be dominant. Next, I move and rotate the sensor. The six sensor channels change, and the model probabilities may also change. If the selected threshold and consecutive-window rule are satisfied, the message says CNN alert/fall probability sustained. The LED and buzzer then activate. This is a functional demonstration, not a clinical test.

16:30–17:35 — Real KFall replay

To make the demonstration repeatable, I created a replay of KFall trial S06T20R01. The original 100 Hz trial was converted to the 50 Hz working rate and placed in replay_data.h. The ESP32 then feeds the recorded six-channel sequence through the same rolling window and the same trained INT8 model. The replay produced a high fall probability and activated the alarm. This confirms the full dataset-to-firmware path.

17:35–18:30 — Main results

The reference deployment model has about 7,947 parameters, an INT8 size of about 22.5 KB, and an estimated tensor arena of about 8 KB. The INT8 result stayed very close to FP32. Cross-dataset testing showed a large and asymmetric generalization gap. KFall transfers better than FallAID. Instance normalization helped, while more expensive adaptation methods did not consistently help.

18:30–19:20 — Benefits and limitations

The benefits are low cost, local processing, no cloud dependency, small model size, and a clear alarm output. The limitations are equally important. Falls in public datasets are simulated. The supplied SisFall release lacks full subject metadata. Physical latency, battery life, peak RAM, and false alarms must still be measured on the real board. Therefore this is a strong research prototype, not yet a certified medical device.

19:20–20:00 — Future work and closing

The next steps are to measure physical latency over 1,000 inferences, measure peak RAM, test battery life, build a wearable enclosure, and evaluate controlled activities with volunteers under safe supervision. Future versions may add wireless notification, event logging, better debounce logic, and more real-world data. To conclude, this project connects public datasets, deep learning, INT8 TinyML, ESP32 firmware, and a real sensor alarm into one complete system. Thank you. I am ready for questions.